

HOSPITAL MANAGEMENT ERP

Phase-Wise Implementation Plan

Spring Boot | PostgreSQL | Microservices

Version 1.0

April 2026

Prepared by: Siance Software

CONFIDENTIAL

Table of Contents

1. Introduction

1.1 Purpose

This document is the definitive implementation plan for the Hospital Management ERP (HM-ERP) system. It translates the functional requirements from the Comprehensive System Documentation into a concrete, phase-wise technical execution roadmap using Spring Boot, PostgreSQL, and a microservices architecture.

The plan covers microservice decomposition, database schemas per service, API contracts, infrastructure design, CI/CD pipelines, sprint-level task breakdowns, team structure, and risk mitigation strategies.

1.2 Reference Documents

- Hospital Management ERP - Comprehensive System Documentation v1.0
- Spring Boot Reference Documentation (3.3.x)
- PostgreSQL 16 Documentation
- HL7 FHIR R4 Implementation Guide
- HAPI FHIR Server Documentation

1.3 Technology Stack Overview

Layer	Technology	Version	Purpose
Language	Java	21 (LTS)	Primary development language with virtual threads support
Framework	Spring Boot	3.3.x	Microservice framework with Spring Cloud ecosystem
Database	PostgreSQL	16.x	Primary relational database per microservice
Cache	Redis	7.x	Distributed caching, session store, pub/sub
Message Broker	Apache Kafka	3.7.x	Async event-driven communication between services
Search	Elasticsearch	8.x	Full-text search (patient, drug, diagnosis search)
Object Storage	MinIO / S3	Latest	Document, image, and DICOM file storage
API Gateway	Spring Cloud Gateway	4.1.x	Request routing, rate limiting, auth
Service Discovery	Spring Cloud Consul / Eureka	Latest	Service registration and discovery
Config Management	Spring Cloud Config	Latest	Centralized configuration server

Auth	Keycloak	24.x	Identity and access management, SSO, RBAC
Containerization	Docker	25.x	Service containerization
Orchestration	Kubernetes	1.29.x	Container orchestration, scaling, service mesh
CI/CD	Jenkins / GitHub Actions	Latest	Build, test, deploy automation
Monitoring	Prometheus + Grafana	Latest	Metrics collection and visualization
Logging	ELK Stack (Elastic, Logstash, Kibana)	8.x	Centralized logging and analysis
Tracing	Jaeger / Zipkin	Latest	Distributed request tracing
FHIR Server	HAPI FHIR	7.x	HL7 FHIR R4 compliance layer
Frontend	React / Angular (recommended)	Latest	SPA frontend (separate repository)

2. Microservice Architecture

2.1 Service Decomposition Strategy

Services are decomposed following Domain-Driven Design (DDD) principles. Each microservice owns its bounded context, has its own PostgreSQL database (database-per-service pattern), and communicates via REST APIs (synchronous) and Kafka events (asynchronous).

2.2 Complete Service Catalog

Service Name	Bounded Context	Port	Database	Phase
hm-gateway	API Gateway & routing	8080	None (stateless)	1
hm-auth-service	Authentication & authorization via Keycloak	8081	keycloak_db	1
hm-config-server	Centralized configuration	8888	Git repo	1
hm-discovery-server	Service registry	8761	None (in-memory)	1
hm-patient-service	Patient registration, MPI, demographics	8100	patient_db	1
hm-appointment-service	Scheduling, tokens, queue management	8101	appointment_db	1
hm-opd-service	OPD consultation, prescriptions, clinical notes	8102	opd_db	1
hm-billing-service	Charge capture, invoicing, payments	8103	billing_db	1
hm-master-data-service	ICD codes, drug master, service catalog	8104	master_db	1
hm-notification-service	SMS, email, push, WhatsApp notifications	8105	notification_db	1
hm-ipd-service	Admission, bed mgmt, discharge, nursing	8200	ipd_db	2
hm-emergency-service	ER triage, trauma, rapid workflows	8201	emergency_db	2
hm-lab-service	LIS - orders, samples, results, QC	8202	lab_db	2
hm-radiology-service	RIS - imaging orders, reports, DICOM	8203	radiology_db	2
hm-pharmacy-service	Drug dispensing, formulary, narcotics	8204	pharmacy_db	2
hm-nursing-service	Vitals, MAR, care plans, assessments	8205	nursing_db	2
hm-ot-service	OT scheduling, surgical	8206	ot_db	2

	safety, anesthesia			
hm-inventory-service	Procurement, stock, GRN, vendor mgmt	8300	inventory_db	3
hm-hr-service	Employee, attendance, leave, payroll	8301	hr_db	3
hm-bloodbank-service	Donor, collection, testing, cross-match, issue	8302	bloodbank_db	3
hm-dietary-service	Diet plans, kitchen, meal tracking	8303	dietary_db	3
hm-insurance-service	TPA, pre-auth, claims, settlement	8400	insurance_db	4
hm-quality-service	Incidents, CAPA, infection control, accreditation	8401	quality_db	4
hm-analytics-service	Reports, dashboards, BI, data warehouse	8402	analytics_db (read replica)	4
hm-patient-portal	Patient-facing portal API	8403	Shares patient_db (read)	4
hm-fhir-service	FHIR R4 facade for interoperability	8404	fhir_db (HAPI)	4

2.3 Inter-Service Communication Patterns

Pattern	When to Use	Implementation
Synchronous REST	Query another service for immediate data (e.g., patient lookup)	Spring WebClient / OpenFeign with circuit breaker (Resilience4j)
Async Events (Kafka)	State changes that other services react to (e.g., patient admitted)	Spring Kafka with Avro/JSON schema registry
Saga Pattern	Distributed transactions (e.g., billing + pharmacy + inventory)	Orchestration-based saga via a saga coordinator service
CQRS	High-read/low-write scenarios (e.g., analytics, dashboards)	Separate read models populated via Kafka consumers
Event Sourcing	Audit-critical flows (e.g., medication administration)	Event store per service, materialized views for queries
API Composition	Aggregate data from multiple services for UI	BFF (Backend for Frontend) in gateway layer

2.4 Shared Libraries

Common functionality is extracted into shared Maven libraries to avoid code duplication:

Library	Contents	Maven Artifact
---------	----------	----------------

hm-common	Base DTOs, exception handling, pagination, audit annotations	com.siance.hm:hm-common
hm-security	JWT filter, RBAC annotations, Keycloak integration	com.siance.hm:hm-security
hm-kafka-common	Event DTOs, Kafka config, dead letter queue handling	com.siance.hm:hm-kafka-common
hm-audit	Audit logging aspect, audit trail entity, Kafka audit publisher	com.siance.hm:hm-audit
hm-fhir-mapper	FHIR R4 resource mappers (Patient, Encounter, Observation)	com.siance.hm:hm-fhir-mapper

3. Project Structure and Conventions

3.1 Repository Strategy

Each microservice lives in its own Git repository within a single GitHub/GitLab organization. A parent BOM (Bill of Materials) repository manages dependency versions across all services.

Repository	Purpose
hm-bom	Parent POM with dependency management, plugin versions, shared profiles
hm-common-libs	Multi-module project for shared libraries (hm-common, hm-security, etc.)
hm-patient-service	Patient registration microservice
hm-{service-name}	One repository per microservice (26 total)
hm-infra	Kubernetes manifests, Helm charts, Terraform scripts, Docker Compose files
hm-frontend	React/Angular SPA frontend application
hm-docs	Architecture decisions (ADRs), API documentation, runbooks

3.2 Standard Service Structure

Every microservice follows this standard Maven project layout:

```
hm-patient-service/  
  pom.xml  
  Dockerfile  
  src/main/java/com/siance/hm/patient/  
    PatientServiceApplication.java  
    config/          (SecurityConfig, KafkaConfig, SwaggerConfig)  
    controller/      (PatientController, VisitController)  
    service/         (PatientService, VisitService)  
    repository/      (PatientRepository, VisitRepository)  
    entity/          (Patient, Visit, PatientContact, Allergy)  
    dto/             (PatientCreateDTO, PatientResponseDTO)  
    mapper/          (PatientMapper using MapStruct)  
    event/           (PatientRegisteredEvent, PatientUpdatedEvent)  
    exception/       (PatientNotFoundException, DuplicatePatientException)  
    validator/       (PatientValidator)  
  src/main/resources/  
    application.yml  
    application-dev.yml  
    application-prod.yml
```

```

db/migration/      (Flyway SQL migration scripts)
src/test/java/     (unit + integration tests)

```

3.3 Naming Conventions

Element	Convention	Example
Package	com.siance.hm.{service}.{layer}	com.siance.hm.patient.controller
Entity class	PascalCase, singular noun	Patient, Admission, LabOrder
Repository	{Entity}Repository	PatientRepository
Service	{Entity}Service / {Feature}Service	PatientService, TriageService
Controller	{Entity}Controller	PatientController
DTO	{Entity}{Action}DTO	PatientCreateDTO, PatientResponseDTO
Kafka Topic	hm.{service}.{event}	hm.patient.registered, hm.lab.result-validated
Database	{service}_db	patient_db, lab_db, billing_db
Table	Lowercase, snake_case, plural	patients, lab_orders, vital_signs
API Path	/api/v1/{resource}	/api/v1/patients, /api/v1/lab-orders
Flyway Migration	V{version}__{description}.sql	V1__create_patients_table.sql

3.4 Key Spring Boot Dependencies (Common)

- spring-boot-starter-web - REST APIs
- spring-boot-starter-data-jpa - JPA/Hibernate for PostgreSQL
- spring-boot-starter-validation - Bean validation (Jakarta)
- spring-boot-starter-security - Security with Keycloak adapter
- spring-boot-starter-actuator - Health, metrics, info endpoints
- spring-kafka - Kafka producer/consumer
- springdoc-openapi-starter-webmvc-ui - Swagger/OpenAPI 3.0 docs
- flyway-core - Database migration
- mapstruct - DTO/Entity mapping
- resilience4j-spring-boot3 - Circuit breaker, retry, rate limiter
- spring-boot-starter-cache + spring-boot-starter-data-redis - Redis caching
- micrometer-registry-prometheus - Prometheus metrics export
- logstash-logback-encoder - JSON structured logging for ELK

4. Database Design (Per-Service Schemas)

4.1 Database-Per-Service Strategy

Each microservice owns its database exclusively. No direct cross-database queries are allowed. Data needed from other services is obtained via REST calls or event-driven replication (materialized views). Each database uses PostgreSQL with the following common features:

- UUID as primary key type (pg-uuid) for all entities
- Audit columns on every table: created_at, updated_at, created_by, updated_by
- Soft delete with deleted_at timestamp (nullable)
- Flyway-managed migrations with version control
- Row-level security (RLS) for multi-tenant deployments
- JSONB columns for extensible/dynamic attributes
- Full-text search using PostgreSQL tsvector for name/text searches

4.2 patient_db Schema

Table	Key Columns	Indexes	Notes
patients	id (UUID PK), uhid (UNIQUE), first_name, last_name, dob, gender, blood_group, photo_url, national_id, status, metadata (JSONB)	uhid, national_id, phone_gin (trigram), name_tsvector	Master patient index; trigram index for fuzzy search
patient_contacts	id, patient_id (FK), type, value, is_primary, is_emergency	patient_id, value	Phone, email, emergency contacts
patient_addresses	id, patient_id (FK), type, line1, line2, city, state, country, pin_code, geo_point	patient_id, pin_code	PostGIS geo_point for location
patient_documents	id, patient_id (FK), doc_type, doc_number, issuing_authority, expiry_date, file_url	patient_id, doc_number	Scanned ID documents stored in MinIO
patient_allergies	id, patient_id (FK), allergen, type, severity, reaction, verified_by, verified_at	patient_id, allergen	Drug/food/environmental allergies
visits	id, patient_id (FK), visit_type, department_code, doctor_id, status, token_number, started_at, ended_at	patient_id, visit_type+status, doctor_id, started_at	All visit types; doctor_id references hr_db via event sync
consents	id, patient_id (FK), visit_id (FK), consent_type, version, status, signed_at, witness_name, document_url	patient_id, consent_type	Treatment, data sharing, research consents
next_of_kin	id, patient_id (FK), name, relationship, phone, address,	patient_id	Required for minors

	is_guardian		
--	-------------	--	--

4.3 appointment_db Schema

Table	Key Columns	Indexes
doctor_schedules	id, doctor_id, day_of_week, start_time, end_time, slot_duration_min, max_patients, location_code, is_active	doctor_id+day_of_week, is_active
schedule_overrides	id, doctor_id, override_date, type (holiday/extra/modified), modified_slots (JSONB)	doctor_id+override_date
appointments	id, patient_uhid, doctor_id, appointment_date, slot_time, type, status, source, cancellation_reason, booked_by	patient_uhid, doctor_id+appointment_date, status
tokens	id, appointment_id (FK), department_code, token_number, counter, status, called_at, seen_at, completed_at	department_code+status, token_number
waiting_queues	id, department_code, date, current_token, avg_wait_minutes, display_config (JSONB)	department_code+date

4.4 opd_db Schema

Table	Key Columns	Indexes
opd_visits	id, visit_id (ext), patient_uhid, doctor_id, department, chief_complaint, vitals_snapshot (JSONB), status	visit_id, patient_uhid, doctor_id
clinical_notes	id, opd_visit_id (FK), note_type (SOAP/free), subjective, objective, assessment, plan, created_by	opd_visit_id, created_by
diagnoses	id, opd_visit_id (FK), icd_code, description, type (primary/secondary/differential), certainty	opd_visit_id, icd_code
prescriptions	id, opd_visit_id (FK), doctor_id, status, pharmacy_status, notes	opd_visit_id, pharmacy_status
prescription_items	id, prescription_id (FK), drug_code, drug_name, dose, frequency, route, duration_days, quantity, instructions, substitution_allowed	prescription_id, drug_code
investigation_orders	id, opd_visit_id (FK), test_code, test_name, urgency, clinical_info, status, result_reference	opd_visit_id, status, test_code
referrals	id, opd_visit_id (FK), from_doctor, to_doctor_or_dept, reason, urgency, clinical_summary, status	opd_visit_id, status

4.5 billing_db Schema

Table	Key Columns	Indexes
-------	-------------	---------

charge_master	id, service_code (UNIQUE), name, cpt_code, department, base_rate, tax_percent, is_active, effective_from	service_code, department, cpt_code
tariffs	id, name, type (cash/insurance/corporate), is_default, effective_from, status	type, status
tariff_rates	id, tariff_id (FK), service_code, rate, discount_percent	tariff_id+service_code
invoices	id, invoice_number (UNIQUE), patient_uhid, visit_id, admission_id, type (OPD/IPD), subtotal, tax, discount, net_amount, paid_amount, balance, status	invoice_number, patient_uhid, visit_id, status
invoice_line_items	id, invoice_id (FK), service_code, description, quantity, unit_rate, amount, tax, discount, department, doctor_id, source_module	invoice_id
payments	id, invoice_id (FK), amount, payment_mode, reference_number, gateway_txn_id, collected_by, collected_at, status	invoice_id, reference_number
advances	id, patient_uhid, admission_id, amount, payment_mode, reference_number, status, adjusted_against_invoice_id	patient_uhid, admission_id, status
refunds	id, invoice_id (FK), payment_id (FK), amount, reason, approved_by, processed_at, refund_mode, status	invoice_id, status

4.6 ipd_db Schema

Table	Key Columns	Indexes
admissions	id, admission_number (UNIQUE), patient_uhid, visit_id, type, admitting_doctor, primary_consultant, bed_id, status, admitted_at, discharged_at, estimated_cost	admission_number, patient_uhid, status, bed_id
beds	id, ward_id (FK), floor, room_number, bed_number, category, status, features (JSONB)	ward_id, status, category
wards	id, name, floor, department_code, total_beds, nurse_station_location	department_code
bed_transfers	id, admission_id (FK), from_bed_id, to_bed_id, reason, transferred_at, authorized_by	admission_id
round_notes	id, admission_id (FK), doctor_id, note_type, assessment, plan, orders (JSONB), recorded_at	admission_id, recorded_at
discharge_summaries	id, admission_id (FK), diagnosis_codes (JSONB), procedures (JSONB), summary_text, discharge_meds (JSONB), instructions, follow_up_date, prepared_by	admission_id

4.7 lab_db Schema

Table	Key Columns	Indexes
test_master	id, test_code (UNIQUE), name, loinc_code, section, sample_type, container, volume_ml, tat_hours, is_active	test_code, loinc_code, section
test_parameters	id, test_id (FK), name, unit, data_type, display_order, reference_ranges (JSONB), critical_values (JSONB)	test_id
test_panels	id, panel_code, name, test_ids (JSONB), price	panel_code
lab_orders	id, order_number (UNIQUE), visit_id, patient_uhid, ordered_by, urgency, clinical_info, status, ordered_at	order_number, patient_uhid, status
lab_order_items	id, lab_order_id (FK), test_code, test_name, status, sample_id (FK)	lab_order_id, status
samples	id, barcode (UNIQUE), lab_order_id (FK), sample_type, collected_by, collected_at, received_at, status, rejection_reason	barcode, lab_order_id, status
lab_results	id, lab_order_item_id (FK), parameter_values (JSONB), tech_validated_by, tech_validated_at, doctor_validated_by, doctor_validated_at, status, report_url	lab_order_item_id, status
critical_alerts	id, lab_result_id (FK), parameter_name, value, notified_to (JSONB), acknowledged_by, acknowledged_at	lab_result_id
quality_controls	id, instrument_code, test_code, qc_level, expected_range (JSONB), observed_value, status, action_taken, performed_by, performed_at	instrument_code, test_code, performed_at

4.8 Additional Database Schemas (Summary)

The following services follow the same patterns. Key tables are listed:

Database	Key Tables
pharmacy_db	drug_master, drug_interactions, pharmacy_locations, pharmacy_stock, dispensings, dispensing_items, narcotic_register, drug_returns
radiology_db	exam_master, radiology_orders, radiology_reports, dicom_study_references, contrast_administrations, radiation_dose_log
nursing_db	nursing_assessments, vital_signs, intake_outputs, medication_administrations, care_plans, care_plan_interventions, shift_handovers, nurse_assignments
emergency_db	er_encounters, triage_assessments, trauma_activations, resuscitation_logs, mlc_registrations, er_dispositions
ot_db	ot_rooms, surgery_requests, surgery_schedules, surgical_teams, anesthesia_records, surgical_safety_checklists, surgical_notes, specimen_tracking
inventory_db	item_master, vendors, rate_contracts, purchase_indents, purchase_orders,

	grn_entries, grn_items, stock_entries, stock_transfers, stock_adjustments
hr_db	employees, departments, designations, attendance_records, leave_types, leave_balances, leave_requests, salary_structures, salary_components, payroll_runs, payslips
bloodbank_db	donors, donor_screenings, blood_bags, blood_components, tti_results, cross_match_requests, blood_issues, transfusion_reactions
dietary_db	diet_types, diet_orders, meal_schedules, meal_preparations, meal_deliveries, nutrition_plans
insurance_db	insurance_companies, tpa_master, patient_policies, pre_auth_requests, claims, claim_items, settlements, rejections
quality_db	quality_indicators, indicator_values, incident_reports, capa_actions, infection_surveillance, audit_schedules, audit_findings, document_controls
analytics_db	fact_patient_visits, fact_revenue, fact_lab_orders, dim_time, dim_department, dim_doctor, dim_diagnosis (Star schema)

5. Phase 1: Foundation (Weeks 1-16)

5.1 Phase 1 Overview

Phase 1 establishes the foundational infrastructure, shared libraries, and core business modules that all subsequent phases depend on. At the end of this phase, the hospital can register patients, book appointments, conduct OPD consultations, write prescriptions, and generate basic billing.

Attribute	Detail
Duration	16 weeks (4 months)
Services Delivered	hm-gateway, hm-auth-service, hm-config-server, hm-discovery-server, hm-patient-service, hm-appointment-service, hm-opd-service, hm-billing-service, hm-master-data-service, hm-notification-service
Team Size	8-10 developers, 1 architect, 1 DevOps, 2 QA, 1 BA, 1 PM
Key Milestone	OPD go-live: patients can register, consult, receive prescriptions, and pay bills

5.2 Sprint Breakdown - Phase 1

Sprint 1-2 (Weeks 1-4): Infrastructure & Shared Libraries

Task	Owner	Deliverable
Set up Git repositories, branching strategy (GitFlow)	DevOps	All repos created with branch protection rules
Create hm-bom parent POM with dependency management	Architect	Parent POM published to Nexus/Artifactory
Build hm-common library (base DTOs, exceptions, pagination, audit)	Backend Lead	Library v1.0 published
Build hm-security library (JWT filter, RBAC annotations, Keycloak)	Security Dev	Library v1.0 published
Build hm-kafka-common (event DTOs, producer/consumer config, DLQ)	Backend Dev	Library v1.0 published
Build hm-audit library (audit aspect, trail entity, Kafka publisher)	Backend Dev	Library v1.0 published
Set up Keycloak with hospital realm, roles, clients	DevOps/ Security	Keycloak configured with all Phase 1 roles
Deploy PostgreSQL instances for Phase 1 services (via Docker)	DevOps	6 PostgreSQL databases running
Deploy Kafka cluster (3-broker), Redis, Elasticsearch	DevOps	Infrastructure stack running
Set up hm-config-server with Git-backed config	Backend Dev	Config server serving all service configs
Set up hm-discovery-server	Backend Dev	Service discovery operational

(Consul/Eureka)		
Set up hm-gateway with routing, rate limiting, CORS	Backend Dev	Gateway routing to all Phase 1 services
Set up CI/CD pipeline (build, test, Docker build, deploy to dev)	DevOps	Pipeline green for all repos
Set up monitoring stack (Prometheus + Grafana + ELK + Jaeger)	DevOps	Dashboards visible for all services
Create Docker Compose for local development	DevOps	docker-compose up starts full stack locally
Set up Kubernetes namespace, Helm chart templates	DevOps	K8s dev namespace with Helm base charts

Sprint 3-4 (Weeks 5-8): Patient Registration & Master Data

Task	Owner	Deliverable
hm-master-data-service: ICD-10 code import and search API	Backend Dev 1	ICD-10 codes loaded, searchable via Elasticsearch
hm-master-data-service: Drug master import (generic, brand, interactions)	Backend Dev 1	Drug database with interaction checking API
hm-master-data-service: Department, service catalog, doctor master	Backend Dev 1	All reference data APIs operational
hm-patient-service: Patient CRUD with UHID generation	Backend Dev 2	Patient registration with auto UHID
hm-patient-service: Patient search (fuzzy name, phone, ID)	Backend Dev 2	Trigram + full-text search working
hm-patient-service: Contact, address, document, allergy management	Backend Dev 2	All sub-entity APIs complete
hm-patient-service: Duplicate detection and merge	Backend Dev 3	Duplicate flagging and admin merge API
hm-patient-service: Visit creation and management	Backend Dev 3	Visit lifecycle APIs
hm-patient-service: Consent capture API	Backend Dev 3	Consent recording with document upload
hm-patient-service: Kafka events (patient.registered, patient.updated)	Backend Dev 2	Events published on patient changes
hm-notification-service: SMS, email, push notification engine	Backend Dev 4	Notification templates + delivery APIs
hm-notification-service: WhatsApp Business API integration	Backend Dev 4	WhatsApp message delivery
Integration testing: patient registration end-to-end	QA	E2E test suite passing
Flyway migrations for patient_db, master_db, notification_db	Backend Devs	All migration scripts versioned and tested

Sprint 5-6 (Weeks 9-12): Appointments & OPD

Task	Owner	Deliverable
hm-appointment-service: Doctor schedule CRUD	Backend Dev 3	Weekly templates + override APIs
hm-appointment-service: Slot availability engine	Backend Dev 3	Real-time available slot calculation
hm-appointment-service: Appointment booking, cancel, reschedule	Backend Dev 3	Appointment lifecycle APIs
hm-appointment-service: Token generation and queue management	Backend Dev 3	Token + queue APIs with wait-time estimation
hm-appointment-service: Appointment reminder events via Kafka	Backend Dev 3	Reminder events consumed by notification-service
hm-opd-service: OPD visit creation and doctor workstation API	Backend Dev 5	Doctor queue + visit data APIs
hm-opd-service: Clinical notes (SOAP) CRUD	Backend Dev 5	Clinical documentation APIs
hm-opd-service: Diagnosis entry with ICD-10 integration	Backend Dev 5	Diagnosis CRUD with code lookup
hm-opd-service: Prescription writing with drug interaction check	Backend Dev 6	Prescription API with safety checks via master-data-service
hm-opd-service: Investigation ordering (lab + radiology placeholders)	Backend Dev 6	Order creation APIs with Kafka events
hm-opd-service: Referral management	Backend Dev 6	Referral CRUD and notification
hm-opd-service: Follow-up scheduling integration	Backend Dev 5	Auto-appointment creation for follow-ups
Integration testing: appointment + OPD workflows	QA	E2E OPD consultation flow tested

Sprint 7-8 (Weeks 13-16): Billing & Go-Live Prep

Task	Owner	Deliverable
hm-billing-service: Charge master and tariff management APIs	Backend Dev 7	Charge master CRUD with multi-tariff support
hm-billing-service: Auto charge capture from OPD events	Backend Dev 7	Kafka consumer for opd.consultation.completed events
hm-billing-service: Invoice generation (OPD)	Backend Dev 7	OPD invoice generation with line items
hm-billing-service: Payment collection (cash, card, UPI)	Backend Dev 8	Payment API with gateway integration
hm-billing-service: Receipt printing API	Backend Dev 8	Configurable receipt templates
hm-billing-service: Advance collection	Backend Dev	Advance deposit management

and adjustment	8	
hm-billing-service: Discount management with approval workflow	Backend Dev 7	Discount request/approve APIs
hm-billing-service: Refund processing	Backend Dev 8	Refund CRUD with authorization
Security hardening: penetration testing, OWASP review	Security Dev	Security report with all critical issues resolved
Performance testing: load test all Phase 1 APIs	QA/DevOps	Performance report, SLA thresholds met
User acceptance testing (UAT)	BA + QA	UAT sign-off from hospital stakeholders
Data migration: existing patient data import	Backend Lead	Migration scripts tested and validated
Staff training: front desk, doctors, billing	BA/PM	Training completed for 50+ users
Go-live preparation: production deployment, monitoring, support	DevOps/ Team	Phase 1 LIVE in production

6. Phase 2: Clinical Core (Weeks 17-32)

6.1 Phase 2 Overview

Attribute	Detail
Duration	16 weeks (4 months)
Services Delivered	hm-ipd-service, hm-emergency-service, hm-lab-service, hm-radiology-service, hm-pharmacy-service, hm-nursing-service, hm-ot-service
Dependencies	Phase 1 services stable and live
Team Growth	+4 developers (12-14 total), +1 QA, +1 clinical BA
Key Milestone	Full clinical operations: IPD admissions, lab/radiology results, pharmacy dispensing, nursing documentation

6.2 Sprint Breakdown - Phase 2

Sprint 9-10 (Weeks 17-20): IPD & Emergency

Task	Owner	Deliverable
hm-ipd-service: Ward and bed master setup APIs	Backend Dev A	Ward/bed hierarchy with JSONB features
hm-ipd-service: Bed availability board API (real-time via WebSocket)	Backend Dev A	Live bed status dashboard data
hm-ipd-service: Admission workflow (planned, emergency, transfer)	Backend Dev A	Admission creation with bed assignment
hm-ipd-service: Bed transfer with auto billing rate update	Backend Dev B	Transfer API with Kafka billing event
hm-ipd-service: Round notes and progress notes CRUD	Backend Dev B	Clinical documentation for inpatients
hm-ipd-service: Discharge workflow (summary, billing, meds, follow-up)	Backend Dev B	Full discharge APIs with event publishing
hm-emergency-service: Rapid registration (minimal dataset)	Backend Dev C	Quick ER registration linked to patient-service
hm-emergency-service: ESI triage assessment	Backend Dev C	Triage scoring with priority classification
hm-emergency-service: ER tracking board API (WebSocket)	Backend Dev C	Real-time patient tracking with color coding
hm-emergency-service: MLC registration and police notification	Backend Dev C	MLC workflow with audit trail
hm-emergency-service: Disposition and ER-to-IPD handoff	Backend Dev C	Seamless transition to admission
Update hm-billing-service: IPD per-day charges, ER charges	Backend Dev 7	Billing Kafka consumers for IPD/ER events

Sprint 11-12 (Weeks 21-24): Lab & Radiology

Task	Owner	Deliverable
hm-lab-service: Test catalog with LOINC, reference ranges, critical values	Backend Dev D	Test master APIs with parameter management
hm-lab-service: Lab order processing from OPD/IPD events	Backend Dev D	Kafka consumer for investigation orders
hm-lab-service: Sample collection with barcode generation	Backend Dev D	Sample lifecycle APIs with barcode
hm-lab-service: Result entry (manual + instrument interface prep)	Backend Dev E	Result CRUD with delta checking
hm-lab-service: Technical and medical validation workflow	Backend Dev E	Two-step validation with role-based access
hm-lab-service: Critical value alerting via Kafka	Backend Dev D	Critical alert events consumed by notification-service
hm-lab-service: Report generation and PDF rendering	Backend Dev E	Lab report template engine
hm-lab-service: QC module for instrument quality tracking	Backend Dev E	Levey-Jennings chart data APIs
hm-radiology-service: Exam master and modality configuration	Backend Dev F	Radiology catalog APIs
hm-radiology-service: Imaging order management	Backend Dev F	Order lifecycle with scheduling
hm-radiology-service: DICOM MWL integration (HL7/DICOM)	Backend Dev F	Modality worklist push to equipment
hm-radiology-service: Reporting (structured + narrative)	Backend Dev F	Report creation and verification workflow
hm-radiology-service: PACS integration (WADO-RS)	Backend Dev F	Image retrieval API via DICOMweb
Update billing: lab and radiology charge capture	Backend Dev 7	Auto-billing from lab/radiology events

Sprint 13-14 (Weeks 25-28): Pharmacy & Nursing

Task	Owner	Deliverable
hm-pharmacy-service: Drug master sync from master-data-service	Backend Dev G	Drug catalog with formulary flags
hm-pharmacy-service: OPD dispensing against prescriptions	Backend Dev G	Prescription queue + dispensing APIs
hm-pharmacy-service: IPD dispensing (unit dose, ward stock)	Backend Dev G	Inpatient medication dispensing
hm-pharmacy-service: Batch/expiry tracking and stock management	Backend Dev G	Real-time stock with FEFO issuing
hm-pharmacy-service: Narcotic register	Backend Dev	Controlled substance workflow

with double verification	G	
hm-pharmacy-service: Return and substitution management	Backend Dev G	Return/substitute APIs with approvals
hm-nursing-service: Initial assessment and care plan CRUD	Backend Dev H	Nursing assessment APIs
hm-nursing-service: Vital signs recording (configurable frequency)	Backend Dev H	Vitals API with trending data
hm-nursing-service: MAR (Medication Administration Record)	Backend Dev H	MAR grid with barcode verification
hm-nursing-service: I/O charting	Backend Dev H	Intake-output recording APIs
hm-nursing-service: Pain, fall risk, pressure ulcer assessments	Backend Dev H	Standardized assessment tool APIs
hm-nursing-service: Shift handover (SBAR format)	Backend Dev H	Structured handoff APIs
Update billing: pharmacy charge capture	Backend Dev 8	Auto-billing from dispensing events

Sprint 15-16 (Weeks 29-32): OT, Integration & Phase 2 Go-Live

Task	Owner	Deliverable
hm-ot-service: OT room master and availability calendar	Backend Dev I	OT scheduling engine
hm-ot-service: Surgery request and approval workflow	Backend Dev I	Surgery request APIs with multi-level approval
hm-ot-service: WHO Surgical Safety Checklist APIs	Backend Dev I	Three-phase checklist (Sign-in, Time-out, Sign-out)
hm-ot-service: Anesthesia record and surgical notes	Backend Dev I	Intra-operative documentation
hm-ot-service: Specimen tracking to lab	Backend Dev I	Specimen handoff with lab-service integration
hm-ot-service: OT utilization analytics	Backend Dev I	Metrics: on-time start, turnaround, cancellations
Lab instrument interfacing (ASTM/LIS2-A2)	Integration Dev	Bi-directional instrument communication
DICOM worklist testing with real modalities	Integration Dev	MWL verified with CT/MRI/X-ray
Cross-service integration testing: full IPD journey	QA Team	Admit-to-discharge E2E test suite
Performance testing for Phase 2 services	QA/DevOps	Load test results within SLA
UAT for IPD, ER, Lab, Radiology, Pharmacy, Nursing, OT	BA + Clinical BA	UAT sign-off from clinical staff

Phase 2 data migration (existing lab catalogs, drug data)	Backend Lead	Migration validated
Phase 2 staff training (nursing, lab, pharmacy, OT)	BA/PM	Training for 100+ clinical users
Phase 2 production deployment and go-live	DevOps/ Team	Phase 2 LIVE

7. Phase 3: Support Services (Weeks 33-44)

7.1 Phase 3 Overview

Attribute	Detail
Duration	12 weeks (3 months)
Services Delivered	hm-inventory-service, hm-hr-service, hm-bloodbank-service, hm-dietary-service
Dependencies	Phase 1 + 2 services stable
Team Size	Same team (12-14 devs), reassigned from completed Phase 2 work
Key Milestone	Complete supply chain, HR operations, blood bank, and dietary management

7.2 Sprint Breakdown - Phase 3

Sprint 17-18 (Weeks 33-36): Inventory & HR

Task	Owner	Deliverable
hm-inventory-service: Item master, vendor master, store locations	Backend Dev A+B	Master data APIs for supply chain
hm-inventory-service: Purchase indent, PO, approval workflow	Backend Dev A	Procurement cycle APIs
hm-inventory-service: GRN with quality inspection and 3-way matching	Backend Dev A	Goods receipt with validation
hm-inventory-service: Stock management (batch, expiry, FIFO/FEFO)	Backend Dev B	Real-time multi-store stock APIs
hm-inventory-service: Stock transfer between stores/departments	Backend Dev B	Inter-store transfer with approval
hm-inventory-service: Auto-reorder alerts and EOQ calculation	Backend Dev B	Alert engine with configurable thresholds
hm-inventory-service: ABC/VED analysis APIs	Backend Dev A	Inventory analytics endpoints
hm-hr-service: Employee master CRUD with document management	Backend Dev C+D	Employee lifecycle APIs
hm-hr-service: Department, designation, org hierarchy	Backend Dev C	Organization structure APIs
hm-hr-service: Attendance (biometric integration), shift management	Backend Dev C	Attendance recording with shift engine
hm-hr-service: Leave management (types, balances, requests, approvals)	Backend Dev D	Leave workflow APIs
hm-hr-service: Payroll engine (salary structure, computation, statutory)	Backend Dev D	Payroll processing with pay slip generation
hm-hr-service: Credentialing (license tracking, CME, renewals)	Backend Dev D	Credential verification APIs

Sprint 19-20 (Weeks 37-40): Blood Bank & Dietary

Task	Owner	Deliverable
hm-bloodbank-service: Donor registration and screening	Backend Dev E+F	Donor lifecycle APIs with deferral management
hm-bloodbank-service: Blood collection with bag barcode tracking	Backend Dev E	Collection workflow with barcode generation
hm-bloodbank-service: Blood grouping, typing, TTI testing	Backend Dev E	Testing workflow with result recording
hm-bloodbank-service: Component preparation and inventory	Backend Dev F	Component separation with expiry tracking
hm-bloodbank-service: Cross-match and blood issue workflow	Backend Dev F	Compatibility testing and issue APIs
hm-bloodbank-service: Transfusion reaction reporting	Backend Dev F	Reaction recording and investigation workflow
hm-bloodbank-service: FIFO issue, wastage tracking, blood camp	Backend Dev E	Optimization and camp management APIs
hm-dietary-service: Diet type master and diet order integration	Backend Dev G	Diet catalog with IPD integration
hm-dietary-service: Meal scheduling and kitchen prep lists	Backend Dev G	Meal generation engine per ward
hm-dietary-service: Meal delivery tracking	Backend Dev G	Delivery status APIs
hm-dietary-service: Nutrition planning and calorie tracking	Backend Dev G	Dietician assessment APIs
Pharmacy-Inventory integration: auto PO from pharmacy reorder	Integration Dev	Cross-service reorder workflow
Billing integration: blood bank charges, implant billing from inventory	Backend Dev 7	New charge capture consumers

Sprint 21-22 (Weeks 41-44): Integration, Testing & Phase 3 Go-Live

Task	Owner	Deliverable
Cross-module integration: inventory consumption from pharmacy, OT, lab	Integration Dev	Automatic stock deduction workflows
HR integration: doctor schedules sync, employee reference in all modules	Backend Dev C	Employee lookup available across services
Blood bank integration: cross-match from IPD/OT, billing auto-capture	Backend Dev F	End-to-end blood request flow
Performance testing: Phase 3 services	QA/DevOps	Load test results within SLA
UAT: inventory, HR, blood bank, dietary workflows	BA + Clinical BA	UAT sign-off from admin and clinical staff

Data migration: vendor data, employee data, blood bank historical data	Backend Lead	Migration validated
Staff training: store managers, HR, blood bank officers, dieticians	BA/PM	Training completed for relevant staff
Phase 3 production deployment and go-live	DevOps/ Team	Phase 3 LIVE

8. Phase 4: Advanced Capabilities (Weeks 45-56)

8.1 Phase 4 Overview

Attribute	Detail
Duration	12 weeks (3 months)
Services Delivered	hm-insurance-service, hm-quality-service, hm-analytics-service, hm-patient-portal, hm-fhir-service
Dependencies	Phase 1-3 services stable
Team Size	Same team with focus shift to analytics and interoperability
Key Milestone	Insurance processing, quality management, executive dashboards, patient portal, FHIR interoperability

8.2 Sprint Breakdown - Phase 4

Sprint 23-24 (Weeks 45-48): Insurance & Quality

Task	Owner	Deliverable
hm-insurance-service: Insurance/TPA company master	Backend Dev A	Insurance master APIs
hm-insurance-service: Patient policy registration and verification	Backend Dev A	Policy management APIs
hm-insurance-service: Pre-authorization workflow	Backend Dev A	Pre-auth request/approve/reject APIs
hm-insurance-service: Claim generation from billing data	Backend Dev B	Auto claim creation from IPD invoices
hm-insurance-service: Claim submission tracking and follow-up	Backend Dev B	Claim lifecycle with status tracking
hm-insurance-service: Settlement reconciliation	Backend Dev B	Settlement matching with billing
hm-quality-service: Quality indicator definitions and data collection	Backend Dev C	Automated indicator collection from clinical events
hm-quality-service: Incident reporting and investigation workflow	Backend Dev C	Anonymous/identified reporting with RCA
hm-quality-service: CAPA tracking	Backend Dev C	Corrective action lifecycle APIs
hm-quality-service: Infection surveillance and antibiogram	Backend Dev D	HAI tracking with antibiogram generation
hm-quality-service: Document control and audit management	Backend Dev D	Policy/SOP versioning with audit scheduling
hm-quality-service: NABH/JCI standard mapping and compliance score	Backend Dev D	Accreditation readiness dashboard data

Sprint 25-26 (Weeks 49-52): Analytics & Patient Portal

Task	Owner	Deliverable
hm-analytics-service: Star schema design and ETL pipeline from Kafka	Backend Dev E	Data warehouse with Kafka consumers
hm-analytics-service: Clinical reports (patient stats, diagnosis, procedures)	Backend Dev E	Clinical analytics APIs
hm-analytics-service: Financial reports (revenue, receivables, collections)	Backend Dev F	Financial dashboard data
hm-analytics-service: Operational reports (bed occupancy, TAT, utilization)	Backend Dev F	Operational metrics APIs
hm-analytics-service: Executive KPI dashboard API	Backend Dev E	CEO/CMO dashboard data endpoints
hm-analytics-service: Ad-hoc report builder API	Backend Dev F	Dynamic query builder for power users
hm-analytics-service: Scheduled report generation and email delivery	Backend Dev E	Cron-based report dispatch
hm-patient-portal: Patient login (OTP/password) and profile	Backend Dev G	Patient self-service auth
hm-patient-portal: Appointment booking and management	Backend Dev G	Online booking via appointment-service
hm-patient-portal: Lab/radiology report download	Backend Dev G	Report access with consent verification
hm-patient-portal: Bill view and online payment	Backend Dev H	Payment gateway integration for patients
hm-patient-portal: Feedback submission	Backend Dev H	Feedback collection and routing
hm-patient-portal: Teleconsultation booking	Backend Dev H	Video call scheduling with doctor calendar

Sprint 27-28 (Weeks 53-56): FHIR, Final Integration & Project Closure

Task	Owner	Deliverable
hm-fhir-service: HAPI FHIR server setup with custom resource providers	Backend Dev I	FHIR R4 server with Patient, Encounter, Observation resources
hm-fhir-service: Patient FHIR resource mapping from patient-service	Backend Dev I	FHIR Patient search and read
hm-fhir-service: Encounter, Condition, MedicationRequest mapping	Backend Dev I	Clinical FHIR resources from OPD/IPD
hm-fhir-service: DiagnosticReport, Observation from lab/radiology	Backend Dev I	Lab/radiology FHIR resources
hm-fhir-service: ABDM integration (India) / national health ID linking	Backend Dev I	National health ecosystem connectivity
End-to-end regression testing: all 26	QA Team	Full regression suite passing

services		
Security audit: comprehensive penetration test	Security Dev	Security audit report, all critical issues resolved
Performance testing: full system under peak load	QA/DevOps	System handles 2000+ concurrent users
Disaster recovery drill	DevOps	DR test successful, RTO/RPO within target
Final UAT for Phase 4 features	BA + Stakeholders	UAT sign-off
Operations runbook and handover documentation	DevOps/ Architect	Complete ops guide
Phase 4 production deployment and system-wide go-live	DevOps/ Team	FULL SYSTEM LIVE
30-day hypercare support	Full Team	Issue resolution within SLA
Project closure and handover to operations	PM	Signed project closure report

9. CI/CD Pipeline Design

9.1 Pipeline Stages

Every microservice follows the same pipeline stages, triggered on each Git push:

Stage	Tools	Actions	Failure Action
1. Code Quality	SonarQube, Checkstyle, SpotBugs	Static analysis, code coverage check (min 80%), bug detection	Block merge, notify developer
2. Unit Tests	JUnit 5, Mockito	Run all unit tests	Block merge
3. Integration Tests	Testcontainers (PostgreSQL, Kafka, Redis)	Run integration tests with real containers	Block merge
4. Build	Maven, Docker	mvn package, docker build, push to registry	Notify team
5. Security Scan	Trivy, OWASP Dependency Check	Container image scan, dependency vulnerability scan	Block deploy if critical CVE
6. Deploy to Dev	Helm, Kubernetes	Deploy to dev namespace	Notify DevOps
7. Smoke Tests	REST Assured, Postman/Newman	Health check, basic API contract tests	Rollback deploy
8. Deploy to Staging	Helm, Kubernetes (manual approval)	Deploy to staging after dev validation	Block further promotion
9. E2E Tests	Cypress / Playwright (UI), REST Assured (API)	Full end-to-end workflow tests	Block production deploy
10. Deploy to Production	Helm, ArgoCD (manual approval)	Rolling update deployment	Auto-rollback on health check failure

9.2 Branching Strategy

Branch	Purpose	Deploys To
main	Production-ready code	Production (via tag/release)
develop	Integration branch for features	Dev environment (auto)
feature/*	Individual feature branches	Dev (on merge to develop)
release/*	Release candidates	Staging (auto), Production (manual)
hotfix/*	Emergency production fixes	Production (fast-track)

9.3 Docker Image Strategy

- Base image: eclipse-temurin:21-jre-alpine (minimal JRE)

- Multi-stage Dockerfile: build stage (Maven) + runtime stage (JRE only)
- Image tagging: {service}:{version}-{git-sha} for traceability
- Image registry: Harbor / AWS ECR / GCP Artifact Registry
- Image scanning on every push with Trivy

10. Infrastructure and Kubernetes Deployment

10.1 Kubernetes Cluster Design

Component	Specification
Cluster Type	Managed Kubernetes (EKS / AKS / GKE) or on-premise K8s via kubeadm/RKE2
Namespaces	hm-dev, hm-staging, hm-production, hm-monitoring, hm-infra
Node Pools	app-pool (application services), db-pool (stateful workloads), monitoring-pool
Ingress Controller	NGINX Ingress with TLS termination (cert-manager for Let's Encrypt)
Service Mesh	Istio (optional, for mTLS, traffic management, observability)
Storage	CSI driver for persistent volumes (PostgreSQL, Kafka, Elasticsearch, Redis)
Secrets Management	Kubernetes Secrets + HashiCorp Vault for sensitive credentials
Auto-scaling	HPA (Horizontal Pod Autoscaler) based on CPU/memory + custom Kafka lag metrics

10.2 Resource Allocation Per Service

Service Category	CPU Request	CPU Limit	Memory Request	Memory Limit	Replicas (Prod)
Infrastructure (gateway, config, discovery)	250m	500m	512Mi	1Gi	2-3
High-traffic (patient, OPD, billing, appointment)	500m	1000m	1Gi	2Gi	3-5
Medium-traffic (IPD, lab, pharmacy, nursing)	500m	1000m	1Gi	2Gi	2-3
Low-traffic (dietary, quality, blood bank)	250m	500m	512Mi	1Gi	2
Stateful (PostgreSQL per service)	1000m	2000m	2Gi	4Gi	Primary + 1 replica
Infrastructure (Kafka, Redis, Elasticsearch)	1000m	2000m	2Gi	8Gi	3 (cluster)

10.3 Production Architecture Summary

Production environment topology:

- Load Balancer (cloud LB or HAProxy) with SSL termination
- NGINX Ingress Controller routing to API Gateway (hm-gateway, 3 replicas)

- API Gateway routes to internal services via Kubernetes service discovery
- Each microservice: 2-5 replicas with HPA, liveness/readiness probes
- PostgreSQL: Primary + streaming replica per service database, pgBouncer for connection pooling
- Kafka: 3-broker cluster with replication factor 3, min ISR 2
- Redis: 3-node Sentinel cluster for caching and session management
- Elasticsearch: 3-node cluster (2 data + 1 master) for search
- MinIO: Distributed mode (4+ nodes) for object storage
- Monitoring: Prometheus (2 replicas) + Grafana + AlertManager + PagerDuty integration
- Logging: Fluentd DaemonSet collecting from all pods into Elasticsearch + Kibana
- Tracing: Jaeger with Elasticsearch backend for distributed trace storage

10.4 Docker Compose for Local Development

A `docker-compose.yml` in the `hm-infra` repository allows developers to spin up the entire infrastructure locally:

- PostgreSQL (single instance with multiple databases)
- Kafka (single broker) + Zookeeper
- Redis (single instance)
- Elasticsearch (single node)
- Keycloak (with pre-configured realm)
- MinIO (single instance)
- Mailhog (email testing)

Individual services are run from IDE or via `mvn spring-boot:run`, connecting to Dockerized infrastructure.

11. Team Structure and Responsibilities

11.1 Core Team Composition

Role	Count	Responsibilities
Technical Architect	1	Architecture decisions, code reviews, cross-service design, ADRs
Backend Lead	1	Shared libraries, code quality standards, mentoring, complex integrations
Senior Backend Developers	4-6	Microservice development, API design, Kafka integration
Mid-Level Backend Developers	4-6	Feature implementation, unit/integration tests, bug fixes
DevOps Engineer	1-2	CI/CD, Kubernetes, infrastructure, monitoring, security
Security Engineer	1	Keycloak setup, penetration testing, security reviews, HIPAA compliance
QA Engineers	2-3	Test automation, E2E testing, performance testing, UAT coordination
Business Analyst (Clinical)	1-2	Clinical workflow validation, stakeholder communication, UAT facilitation
Project Manager	1	Sprint planning, risk management, stakeholder reporting, go-live coordination
UI/UX Designer	1	Screen designs, component library, user research
Frontend Developers	2-3	React/Angular SPA, responsive design, component development

11.2 Team Scaling by Phase

Phase	Backend	Frontend	QA	DevOps	BA/PM	Total
Phase 1 (Foundation)	8	2	2	2	2	16
Phase 2 (Clinical Core)	12	3	3	2	3	23
Phase 3 (Support Services)	10	3	2	1	2	18
Phase 4 (Advanced)	10	3	3	2	2	20

12. Testing Strategy

12.1 Testing Pyramid

Level	Scope	Tools	Coverage Target	Run Frequency
Unit Tests	Individual classes, methods, business logic	JUnit 5, Mockito, AssertJ	80%+ line coverage	Every commit
Integration Tests	Service + database + Kafka + Redis	Testcontainers, Spring Boot Test	All repository and Kafka flows	Every commit
Contract Tests	API compatibility between services	Spring Cloud Contract / Pact	All inter-service APIs	Every PR
E2E Tests (API)	Full workflow across multiple services	REST Assured, Karate	Critical patient journeys	Nightly + pre-release
E2E Tests (UI)	Frontend workflows	Cypress / Playwright	Core user flows	Nightly
Performance Tests	Load, stress, endurance	Gatling / k6	SLA thresholds	Weekly + pre-release
Security Tests	OWASP Top 10, dependency scan	OWASP ZAP, Trivy, Snyk	Zero critical/high vulns	Every build + quarterly pentest
Chaos Tests	Resilience under failure	Chaos Monkey / Litmus	Service recovery within SLA	Monthly

12.2 Key Test Scenarios

The following critical clinical workflows must have comprehensive E2E test coverage:

1. Complete OPD journey: register patient, book appointment, consult, prescribe, order labs, bill, pay
2. Full IPD journey: admit, assign bed, nursing assessment, doctor rounds, lab/radiology orders, results, surgery, pharmacy, discharge, final bill
3. Emergency flow: rapid registration, triage, treatment, disposition (admit/discharge/transfer)
4. Lab lifecycle: order received, sample collected, results entered, validated, critical alert, report generated
5. Pharmacy: prescription received, drug interaction check, dispense with barcode, narcotic verification
6. Blood bank: donor screen, collect, test, prepare components, cross-match, issue, transfusion reaction
7. Insurance: pre-auth request, approval, IPD treatment, claim generation, submission, settlement
8. Cross-service billing: charges from OPD + lab + radiology + pharmacy automatically captured in invoice

13. Monitoring and Observability

13.1 Three Pillars

Pillar	Tool	What We Track
Metrics	Prometheus + Grafana	JVM metrics (heap, GC, threads), API latency (p50/p95/p99), request rate, error rate, Kafka consumer lag, DB connection pool, cache hit ratio
Logging	ELK Stack (Fluentd + Elasticsearch + Kibana)	Structured JSON logs with correlation ID, request/response logging at gateway, error stack traces, audit events
Tracing	Jaeger / Zipkin via Micrometer Tracing	Distributed request traces across services, bottleneck identification, dependency maps

13.2 Alerting Rules

Alert	Condition	Severity	Notification
Service Down	Health check failing for > 30 seconds	Critical	PagerDuty + SMS + Slack
High Error Rate	> 5% error rate over 5 minutes	Critical	PagerDuty + Slack
High Latency	p95 latency > 2 seconds for 5 minutes	Warning	Slack + Email
Database Connection Pool Exhausted	Active connections > 90% of max	Critical	PagerDuty + Slack
Kafka Consumer Lag	Lag > 10000 messages for 10 minutes	Warning	Slack
Disk Usage	> 85% disk utilization	Warning	Slack + Email
Certificate Expiry	TLS cert expires within 14 days	Warning	Email + Slack
Failed Login Attempts	> 10 failed logins for same user in 5 minutes	Security	Security team + Slack

13.3 Grafana Dashboard Catalog

- System Overview: all services health, request rates, error rates at a glance
- Per-Service Dashboard: JVM metrics, API metrics, DB metrics for each microservice
- Kafka Dashboard: topic throughput, consumer lag, partition distribution
- PostgreSQL Dashboard: query performance, connection pool, replication lag, table sizes
- Business Dashboard: patient registrations/day, appointments, lab orders, revenue (pulled from analytics-service)
- Security Dashboard: failed logins, blocked requests, certificate status

14. Risk Management

14.1 Technical Risks

Risk	Probability	Impact	Mitigation Strategy
Microservice complexity leads to distributed debugging challenges	High	Medium	Invest in observability (Jaeger tracing, correlation IDs) from Day 1; train team on distributed debugging
Data consistency issues across service boundaries	High	High	Use saga pattern for transactions, event sourcing for audit-critical flows, idempotent consumers
Kafka message ordering and delivery guarantees	Medium	High	Partition by entity ID, use exactly-once semantics where available, dead letter queues
Database schema migration conflicts in CI/CD	Medium	Medium	Flyway with strict versioning, migration review as part of PR process
Performance degradation under peak hospital load	Medium	High	Regular load testing (weekly), HPA auto-scaling, Redis caching, database query optimization
Lab instrument interface compatibility issues	High	Medium	Early POC with hospital instruments, ASTM middleware, fallback to manual result entry
DICOM/PACS integration complexity	Medium	Medium	Use proven libraries (dcm4che), early integration testing with hospital PACS
Keycloak single point of failure	Low	Critical	Keycloak HA cluster (2+ nodes), database replication, session persistence in Redis

14.2 Project Risks

Risk	Probability	Impact	Mitigation Strategy
Clinical staff resistance to digital workflows	High	High	Clinical BA embedded in team, super-user program, gradual rollout, parallel run period
Scope creep from hospital stakeholders	High	Medium	Strict change control board, MoSCoW backlog, phase gates with sign-off
Key developer attrition mid-project	Medium	High	Knowledge sharing sessions, pair programming, comprehensive documentation, bus factor > 1 per service
Regulatory compliance gaps discovered late	Medium	High	Compliance review at each phase gate, external audit at Phase 2 completion
Integration with legacy hospital systems (HIS, LIS)	High	Medium	Early discovery of existing systems, HL7/FHIR adapter approach, API-first integration
Go-live disruption to hospital operations	Medium	Critical	Phased go-live, parallel run, 24/7 war room during go-live week, instant rollback plan

15. Overall Timeline and Milestones

Milestone	Target Date	Deliverables	Go/No-Go Criteria
Project Kickoff	Week 1	Team onboarded, repos created, infra provisioned	All team members active, dev environment working
Infrastructure Ready	Week 4	K8s cluster, CI/CD, monitoring, shared libs v1.0	Pipeline green for all repos, monitoring dashboards live
Phase 1 Go-Live	Week 16	Patient, Appointment, OPD, Billing live	UAT passed, performance within SLA, zero critical bugs
Phase 2 Go-Live	Week 32	IPD, ER, Lab, Radiology, Pharmacy, Nursing, OT live	UAT passed, instrument interfaces working, clinical sign-off
Phase 3 Go-Live	Week 44	Inventory, HR, Blood Bank, Dietary live	UAT passed, data migration validated, admin sign-off
Phase 4 Go-Live (Full System)	Week 56	Insurance, Quality, Analytics, Portal, FHIR live	Full regression passed, security audit clean, DR drill successful
Hypercare Complete	Week 60	30-day post go-live support	Issue backlog under control, SLA met for 30 days
Project Handover	Week 62	Operations runbook, training materials, warranty period	Signed handover and closure documents

15.1 Estimated Effort Summary

Phase	Duration	Team (avg)	Person-Months	Key Complexity
Phase 1: Foundation	4 months	16	64	Architecture setup, shared libraries, auth, core patient flows
Phase 2: Clinical Core	4 months	23	92	Clinical workflows, instrument integration, DICOM, real-time nursing
Phase 3: Support Services	3 months	18	54	Supply chain, payroll, blood bank, cross-module integration
Phase 4: Advanced	3 months	20	60	Analytics/BI, FHIR interoperability, patient portal, insurance
Total	14 months	~19 avg	~270	26 microservices, full hospital digitization

END OF DOCUMENT

Hospital Management ERP - Phase-Wise Implementation Plan v1.0

Prepared by Siance Software | April 2026